

SAiDL Summer 2026 Induction Assignment

Core ML: Long-Context Sequence Modelling
&
Mechanistic Interpretability:
Quantisation and Representation Geometry

Society for Artificial Intelligence and Deep Learning
June 7, 2026

Part I

Core ML: Long-Context Sequence Modelling

1 Introduction and Motivation

Modern language models are now expected to work with much longer prompts, not just short snippets of text. That makes long-context modelling a practical bottleneck: the model has to keep useful information alive across thousands of tokens while still being fast enough and small enough to run on limited hardware.

Transformers sit at the centre of this problem. Their attention mechanism is powerful, but its cost grows quickly with sequence length, so every design choice matters. In this part of the report, we compare attention variants, positional encodings, and hybrid blocks to see how each one changes the balance between perplexity, memory use, speed, and scaling behaviour.

The study is organised around three mostly independent design choices:

1. **Attention mechanism** — how tokens attend to each other;
2. **Positional encoding** — how position information is injected;
3. **Architectural block** — whether local convolutional context-mixing is combined with attention.

The `core_ml/` codebase keeps these choices configurable through Hydra, so experiments can be changed from YAML files rather than by editing model code.

1.1 Architecture Overview

The base model is a Pre-LayerNorm Transformer decoder in the GPT-2 style:

$$\begin{aligned} \mathbf{x}_0 &= \text{Embed}(\mathbf{t}) + \text{PosEnc}(\mathbf{t}), \\ \mathbf{x}_{\ell+1} &= \mathbf{x}_\ell + \text{Attn}(\text{LN}(\mathbf{x}_\ell)) + \text{FFN}(\text{LN}(\cdot)), \quad \ell = 0, \dots, L-1, \\ \hat{\mathbf{y}} &= \text{LM_Head}(\text{LN}(\mathbf{x}_L)). \end{aligned}$$

- **Weight Tying:** The token embedding and LM head share weights, which cuts down the parameter count and keeps the input/output representations aligned. Since both layers use the same matrix, one call to `nn.init.normal_(token_embedding.weight, std=0.02)` initialises both.
- **Weight Initialization:** Linear layers and embeddings follow the GPT-2 initialization scheme, $\mathcal{N}(0, 0.02)$.
- **Scaled Residual Initialization:** The attention `out_proj` and the second FFN linear layer use $\mathcal{N}(0, 0.02/\sqrt{2L})$, which helps keep residual-stream variance from growing with depth.

Default hyperparameters (from `core_ml/configs/model/transformer.yaml`):

Parameter	Value
d_{model}	256
L (layers)	6
H (heads)	4
d_{ff}	1024
Vocabulary	50 264 (distilgpt2 tokeniser)
Dropout	0.05

2 Dataset and Loading Strategy

2.1 Dataset: WikiText-2

All experiments use WIKITEXT-2 (`Salesforce/wikitext`, `wikitext-2-raw-v1`) through the HuggingFace `datasets` library. The dataset is small enough for fast iteration, but still large and structured enough to make architectural differences visible. Its predefined train, validation, and test splits also make comparisons easier to keep consistent.

In this project, WIKITEXT-2 acts as a controlled benchmark for testing how attention mechanisms, positional encodings, and block-level changes affect language-modelling performance.

Property	Value
Dataset	WikiText-2
Source	Wikipedia articles
Tokeniser	DistilGPT-2 BPE
Vocabulary size	50 264
Training tokens (approx.)	2.1M
Validation tokens (approx.)	217K

2.2 Tokenisation and Data Processing

The data pipeline lives in `core_ml/train/dataset.py`. It loads the train and validation splits from HuggingFace, tokenises each document with the DistilGPT-2 tokenizer, and then drops the raw text once token IDs have been produced.

Instead of padding and batching documents one by one, the pipeline concatenates all tokenised documents in a split into one long stream. This keeps the setup simple, reduces wasted padding, and gives the model longer uninterrupted stretches of text to learn from.

That stream is then cut into chunks of `seq_len + 1` tokens. The first `seq_len` tokens become the input, and the same chunk shifted by one token becomes the target. This is the usual next-token prediction setup for autoregressive language modelling.

The final step wraps the chunks in PyTorch DataLoaders for training and validation. Pinned memory and configurable worker counts are used where possible to keep loading from becoming the bottleneck.

2.3 Training Setup

All models use the same WIKITEXT-2 training setup, defined in `core_ml/configs/training/default.y`.

Hyper-parameter	Value
Optimiser	AdamW
Learning rate	0.01
Minimum learning rate	0.003
(β_1, β_2)	(0.9, 0.95)
Weight decay	0.10
Gradient clip	1.0
Gradient accumulation steps	2
Warm-up iterations	200
Scheduler	cosine with warm-up
Mixed precision	FP16
Epochs	20
Early stopping	2 000 steps
Evaluation interval	200 steps
Logging interval	100 steps
Seed	42

- Experiments were run on a Google Colab T4 GPU, so the priority was to compare many designs fairly rather than fully tune one model.
- Gradient clipping at 1.0 and FP16 mixed precision helped keep training stable while improving memory use and throughput on the available hardware.
- Each variant used the same 2 000-step budget to keep the comparison fair.
- Batch size was adjusted by sequence length using $\{512:8, 1024:4, 2048:2, 4096:1\}$ so GPU memory stayed roughly comparable across runs.
- **Experiment Tracking:** All runs and metrics were logged to W&B, which made it easier to monitor training and compare configurations side by side.

The experiment runner notebook (`experiments_runner.ipynb`) calls the training entry-point (`core_ml/train/train.py`).

3 Baseline Transformer

3.1 Vanilla Multi-Head Attention

Vanilla Multi-Head Attention (MHA), introduced with the original Transformer, is the baseline used for every comparison in this study.

3.1.1 Mechanism

For an input sequence $\mathbf{X} \in \mathbb{R}^{N \times d}$, one attention head computes:

$$\mathbf{Q}_h = \mathbf{X}\mathbf{W}_h^Q, \quad \mathbf{K}_h = \mathbf{X}\mathbf{W}_h^K, \quad \mathbf{V}_h = \mathbf{X}\mathbf{W}_h^V, \quad (1)$$

$$\text{Attn}_h(\mathbf{X}) = \text{softmax}\left(\frac{\mathbf{Q}_h\mathbf{K}_h^\top}{\sqrt{d_k}} + \mathbf{M}_{\text{causal}}\right) \mathbf{V}_h, \quad (2)$$

where $d_k = d/H$ is the per-head dimension, $\mathbf{W}_h^Q, \mathbf{W}_h^K, \mathbf{W}_h^V \in \mathbb{R}^{d \times d_k}$ are learned projections, and $\mathbf{M}_{\text{causal}}$ is the upper-triangular causal mask that prevents each position from attending to future tokens:

$$(\mathbf{M}_{\text{causal}})_{ij} = \begin{cases} 0 & j \leq i, \\ -\infty & j > i. \end{cases} \quad (3)$$

The H heads are concatenated and projected back to the residual stream dimension via $\mathbf{W}^O \in \mathbb{R}^{d \times d}$:

$$\text{MHA}(\mathbf{X}) = [\text{Attn}_1; \dots; \text{Attn}_H] \mathbf{W}^O. \quad (4)$$

The implementation is in `core_ml/models/attention/vanilla_attention.py`.

3.2 Implementation Details

The model code keeps positional choices separate from the rest of the Transformer block. In `core_ml/models/blocks.py`, `TransformerBlock` can receive optional `alibi` or `rope` components, but the sinusoidal baseline leaves both slots empty by passing `None`. As a result, the baseline does not pay for any unused positional machinery, while the same block can still be reused for ALiBi or RoPE by changing only what gets passed into it.

This makes the baseline a useful reference point rather than a one-off special case. Attention, positional encoding, and block structure can each be changed independently, so later experiments can isolate one design choice at a time. In that sense, the baseline is the “zero-config” route through the system: standard MHA, sinusoidal positional encoding, and no extra positional mechanism injected into the block.

3.3 Results

Table 1: Baseline Vanilla MHA + Sinusoidal PE at context length 1024.

Run	Val Loss	Val PPL	Throughput (tok/s)	Peak GPU (MB)
Vanilla MHA + Sin	5.74	311.549	105 902	2317.40723

3.4 Discussion

- The baseline was still improving when the fixed training budget ended, so its perplexity should be read as a compute-limited reference point rather than a fully converged result.
- Gradient norms stayed below 1 throughout training (final value ≈ 0.56), which suggests that the optimisation setup was stable despite the aggressive schedule.

4 Attention Variants

Following the assignment specification, this section replaces standard self-attention with alternative attention mechanisms and evaluates them across multiple context lengths. The required metrics are validation perplexity, peak GPU memory consumption, training stability, and inference latency/throughput. The implemented variants include Sliding Window Attention, Grouped Query Attention (GQA), and Softmax-free ReLU Attention.

4.1 Mechanisms

4.1.1 Sliding Window (Local) Attention

Full attention builds an $N \times N$ score matrix, so its time and memory cost scale as $\mathcal{O}(N^2)$. In long sequences, many of those scores involve distant tokens that often add little useful information. Sliding Window Attention keeps each query at position i focused on only the W most recent keys, reducing the ideal attention cost to $\mathcal{O}(N \cdot W)$:

$$\text{mask}_{i,j} = \begin{cases} 1 & \text{if } 0 \leq i - j < W, \\ 0 & \text{otherwise.} \end{cases} \quad (5)$$

Positions outside the window are filled with $-\infty$ before the Softmax, so they receive zero weight. The implementation exposes the window size as a configuration choice; the verified results below correspond to the saved `sliding_window_L1024` checkpoint evaluated at multiple test lengths.

4.1.2 Grouped Query Attention (GQA)

In standard MHA, every query head has its own key and value head. During inference, that makes the KV cache grow as $\mathcal{O}(L \cdot N \cdot H \cdot d_k)$ across layers, sequence length, heads, and head dimension. Grouped Query Attention reduces this cost by splitting the H query heads into G groups, with each group sharing one KV head:

$$\mathbf{K}^{(g)}, \mathbf{V}^{(g)} \in \mathbb{R}^{N \times d_k}, \quad g = 1, \dots, H_{\text{kv}}, \quad H_{\text{kv}} = H/G. \quad (6)$$

The KV heads are broadcast to all query heads in their group before the dot product, using `einops.repeat`:

```
1 k = repeat(k, "b g s d -> b (g r) s d", r=self.num_groups)
2 v = repeat(v, "b g s d -> b (g r) s d", r=self.num_groups)
```

This cuts KV-cache memory by a factor of G . The trade-off is that query heads within the same group must share the same key/value view of the context.

We test two configurations:

- $H_{\text{kv}} = 2$ ($G = 2$, half the KV heads): a balanced trade-off.
- $H_{\text{kv}} = 1$ ($G = 4$, Multi-Query Attention): maximum KV sharing.

4.1.3 ReLU Attention (Softmax-free)

Softmax turns every attention row into a probability distribution, so each query must put weight somewhere even when all scores are poor. ReLU Attention replaces Softmax with a

ReLU, allowing genuine structural sparsity: if every dot product is negative, the whole attention row becomes zero and contributes nothing to the output:

$$\mathbf{A} = \frac{\text{ReLU}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d_k})}{N}. \quad (7)$$

4.2 Evaluation Protocol

The attention variants are evaluated with the following protocol:

- **Shared architecture:** $d = 256$, $L = 6$, $H = 4$, $d_{\text{ff}} = 1024$, `dropout = 0.05`.
- **Shared training config:** 2000-step budget, AdamW with cosine warm-up, FP16, seed 42 (from `core_ml/utils/helpers.py:set_seed`).
- **Positional encoding:** Sinusoidal for all attention variants in this section (to isolate the attention axis). ALiBi is used for hybrid runs.
- **Context lengths:** $N \in \{512, 1024, 2048, 4096\}$.
- **Batch sizes:** $\{512:8, 1024:4, 2048:2, 4096:1\}$ to maintain a roughly constant GPU memory footprint.
- **Metrics recorded:** validation perplexity (via `core_ml/utils/metrics.py:calculate_perplexity`), peak GPU memory (MB, via `torch.cuda.max_memory_allocated`), inference throughput (tokens/sec, timed over the validation loop), and training stability (standard deviation of the validation loss over the final 300 logged steps).

4.3 Results

Table 2: Validation perplexity ($\downarrow$) for verified attention runs. Both checkpoints were trained at context length 1024 and evaluated at multiple test lengths.

Variant	$L_{\text{test}}=512$	$L_{\text{test}}=1024$	$L_{\text{test}}=2048$	$L_{\text{test}}=4096$
Vanilla MHA	1928.25	311.549	1928.15	1930.21
Sliding Window	334.02	321.70	388.15	805.03
GQA	329.81	321.96	519.62	1507.51
ReLU Attention	51839.68	51852.99	52164.11	52295.94

Note: The ReLU Attention checkpoint was not found during evaluation, so the reported ReLU numbers come from a randomly initialised model and should not be compared as a trained attention variant.

Table 3: Inference throughput (tokens/sec) for verified attention runs.

Variant	$L_{\text{test}}=512$	$L_{\text{test}}=1024$	$L_{\text{test}}=2048$	$L_{\text{test}}=4096$
Vanilla MHA	95 559	129 438	67 652	48 282
Sliding Window	87 221	126 113	65 219	46 198
GQA	97 719	122 542	64 886	46 570
ReLU Attention	105 934	99 298	79 524	54 958

Table 4: Inference latency (ms) for verified attention runs. Latency is reported per evaluation batch from the evaluation logs.

Variant	$L_{\text{test}}=512$	$L_{\text{test}}=1024$	$L_{\text{test}}=2048$	$L_{\text{test}}=4096$
Vanilla MHA	24.7	59.8	161.9	513.8
Sliding Window	24.1	58.1	157.2	489.1
GQA	24.8	60.4	162.1	516.3
ReLU Attention	27.7	68.2	182.9	580.3

Table 5: Peak GPU memory for verified attention runs. Values are taken directly from the evaluation logs.

Variant	$L_{\text{test}}=512$	$L_{\text{test}}=1024$	$L_{\text{test}}=2048$	$L_{\text{test}}=4096$
Vanilla MHA	1246 MB	2317 MB	3718 MB	7012 MB
Sliding Window	1396 MB	2320 MB	3868 MB	7161 MB
GQA	1392 MB	2313 MB	3864 MB	7157 MB
ReLU Attention	1246 MB	2070 MB	3718 MB	7012 MB

Table 6: Training-stability summary for attention variants from the uploaded run history.

Variant	Stability metric	Notes
Vanilla MHA	Final grad norm = 1.07	Last-5 val-loss std = 0.063
Sliding Window	Final grad norm = 0.69	Last-5 val-loss std = 0.052
GQA	Final grad norm = 0.61	Last-5 val-loss std = 0.053
ReLU Attention	–	No trained run in uploaded history

4.4 Discussion

The uploaded run history supplies final $L_{\text{test}} = 1024$ metrics and stability summaries for Vanilla MHA, Sliding Window, and GQA. The comparison should still be read as partial across context lengths because the trained ReLU run is absent and the off-length columns come from the earlier evaluation logs rather than this upload.

5 Positional Encodings

Position information is especially important in the long-context setting because the model has to tell not only what tokens are present, but where they appear relative to one another. Here, we compare four choices—sinusoidal encoding, RoPE, ALiBi, and learned relative bias—and test whether a model trained on shorter sequences can still behave sensibly when evaluated on longer ones.

5.1 Implementations

5.1.1 Sinusoidal (Baseline)

Sinusoidal encoding is the simplest reference point. It gives every position a fixed, non-trainable vector and adds that vector to the token embedding before the sequence enters the Transformer. Because the pattern is deterministic rather than learned, the model can be evaluated at positions it did not explicitly see during training :

$$\text{PE}(p, 2i) = \sin\left(\frac{p}{10000^{2i/d}}\right), \quad \text{PE}(p, 2i + 1) = \cos\left(\frac{p}{10000^{2i/d}}\right). \quad (8)$$

In the code, this is implemented in `core_ml/models/positional/Sinusoidal.py`. The sinusoidal table is built once up to `max_len = 4096`, stored as a non-trainable buffer, and sliced to match the current sequence length during the forward pass. This keeps the baseline cheap and predictable: positional information is added once at the input, with no extra learned parameters.

5.1.2 Rotary Positional Embedding (RoPE)

RoPE moves the positional signal into attention itself. Instead of adding a position vector to the residual stream, it rotates each query and key by an angle determined by its position. This makes the dot product naturally depend on relative offset, which is useful when the same pattern may appear at different places in a long sequence . After attention heads are split, RoPE applies:

$$\mathbf{q}_m \leftarrow \mathbf{q}_m \odot e^{im\theta}, \quad \mathbf{k}_n \leftarrow \mathbf{k}_n \odot e^{in\theta}, \quad (9)$$

so $\mathbf{q}_m^\top \mathbf{k}_n$ depends on the distance $(m - n)$ rather than only on the absolute positions m and n . Implementation: `core_ml/models/positional/Rope.py`. The cosine and sine values used for the rotations are cached as buffers, and the cache can grow if inference uses a sequence longer than `max_seq_len`.

5.1.3 ALiBi (Attention with Linear Biases)

ALiBi adds a fixed, head-specific linear penalty to attention scores before the Softmax :

$$\text{scores}_{h,i,j} \leftarrow \text{scores}_{h,i,j} - m_h \cdot |i - j|, \quad (10)$$

where $m_h = 2^{-8h/H}$. Unlike sinusoidal encoding, ALiBi does not add a separate position vector to the token embeddings. The `Transformer` wrapper therefore uses `nn.Identity()` for `positional_encoding`, leaving position information to enter through the attention bias instead. Implementation: `core_ml/models/positional/Alibi.py`.

5.1.4 Learned Relative Positional Bias

Learned relative bias gives attention a small learned preference for how far apart two tokens are. Instead of storing a separate value for every possible distance, the offset is clipped to a fixed range, $[-D, D]$ with $D = 128$, following Shaw et al. :

$$\text{scores}_{i,j} \leftarrow \text{scores}_{i,j} + \mathbf{E}_{\text{rel}}[\text{clip}(j - i, -D, D) + D]. \quad (11)$$

So, nearby and moderately faraway tokens can get distinct learned biases, while very large distances are grouped into the same edge buckets. In this implementation the table has $2D + 1 = 257$ distance buckets, with one value per attention head in each bucket, initialised from $\mathcal{N}(0, 0.02)$. The resulting bias is then added to the attention scores through the same bias pathway used for ALiBi. Implementation: `core_ml/models/positional/RelativeBias.py`.

5.2 Extrapolation Protocol

Each encoding is trained with Vanilla MHA at $L_{\text{train}} = 512$ tokens. The same checkpoint is then evaluated, without fine-tuning, at $L_{\text{test}} \in \{512, 1024, 2048\}$ to test length extrapolation.

Table 7: Positional extrapolation results: validation perplexity ($\downarrow$) as a function of test length. All models trained at $L_{\text{train}} = 512$.

Encoding	$L_{\text{test}}=512$	$L_{\text{test}}=1024$	$L_{\text{test}}=2048$	$\Delta\text{PPL} (512 \rightarrow 2048)$
Sinusoidal	277.85	495.00	3688.43	3410.58
RoPE	151.43	144.33	169.77	18.34
ALiBi	182.13	160.41	159.23	-22.90
Relative Bias	177.27	206.98	285.84	108.57

6 Convolution + Attention Hybrids

The next set of experiments adds lightweight 1D convolutions to the best attention and positional-encoding choices from the earlier sections.

6.1 Architectures

6.1.1 Conv-Before-Attention (conv_before_attn)

This block inserts a depthwise-separable causal Conv1D layer before attention, giving the model a cheap local n -gram summary before it performs global mixing:

$$\mathbf{x} \rightarrow \underbrace{\text{DWSepConv}(\text{LN}(\mathbf{x}))}_{\text{local}} \rightarrow \underbrace{\text{Attn}(\text{LN}(\cdot))}_{\text{global}} \rightarrow \text{FFN}(\text{LN}(\cdot)). \quad (12)$$

Kernel size: 3. Implementation: `ConvBeforeAttnBlock` in `core_ml/models/hybrid/hybrid_blocks.py`

6.1.2 Gated Convolutional FFN (gated_conv_ffn)

This variant replaces the standard MLP feedforward block with a SwiGLU-gated, depthwise-separable convolutional feedforward layer, inspired by Conformer :

$$\text{output} = \text{Proj}_{\text{out}}(\text{DWConv}(\text{SiLU}(\mathbf{W}_{\text{gate}}\mathbf{x}) \odot \mathbf{W}_{\text{val}}\mathbf{x})). \quad (13)$$

The gate controls which local patterns are allowed through. Implementation: `GatedConvFFNBlock` in `core_ml/models/hybrid/hybrid_blocks.py`.

6.2 Results

Table 8: Hybrid architecture comparison. Perplexity ($\downarrow$), throughput ($\uparrow$), and peak GPU memory ($\downarrow$) at $N=1024$.

Architecture	PPL@1024	Tok/s	Latency (ms)	Peak GPU (MB)	Δ PPL vs. Pure
Pure Attention	311.549	129437.86	7.91	2317.41	0.00
Conv-Before-Attn (fixed)	166.9861	71973.34	14.23	2616.07	-144.5629
Gated Conv FFN (fixed)	217.99	76584.06	13.37	2577.61	-93.559

Note: The uploaded history contains final validation metrics at $N = 1024$ only. Latency is estimated as $1024/\text{tok/s}$.

7 Consolidated Results and Comparative Analysis

7.1 Full Comparison Table

Table 9: Full experiment matrix: validation perplexity, throughput, and peak GPU memory for completed runs.

Attention	PE	PPL@1024	PPL@2048	Tok/s	Peak (MB)
GQA ($H_{kv}=1$)	Sinusoidal	321.70	347.16	26664.02	3313.40
GQA ($H_{kv}=2$)	Sinusoidal	321.96	343.47	122541.66	2312.61
Sliding Window ($W=N/2$)	Sinusoidal	313.27	355.68	26332.31	3326.50
Sliding Window ($W=N/4$)	Sinusoidal	321.70	357.19	126113.12	2320.47
Sliding Window ($W=N/8$)	Sinusoidal	327.17	345.44	26917.23	3326.50
Vanilla	ALiBi	—	—	24734.56	3347.18
Vanilla	Relative Bias	—	—	20698.07	3359.13
Vanilla	RoPE	—	—	25020.06	3323.46
Vanilla	Sinusoidal	311.549	350.20	129437.86	2317.41
ReLU	Sinusoidal	—	—	—	—
SW + Conv-Before-Attn	ALiBi	166.9861	—	71973.34	2616.07
SW + Gated Conv FFN	ALiBi	217.99	—	76584.06	2577.61
SW + Interleaved	ALiBi	—	—	—	—

Note: Rows present in the uploaded run history use final validation throughput and peak GPU values at $N = 1024$; rows absent from the upload either retain earlier benchmark values or are marked with “—”.

Part II

Mechanistic Interpretability: Quantisation and Representation Geometry

8 Introduction and Motivation

Large language models can solve impressive tasks, but their internal representations are still hard to read. Mechanistic interpretability tries to make those hidden computations less opaque: instead of treating the model as a black box, it asks what features the model builds, where those features live, and how they influence behaviour.

In this part of the report, the goal is to look past the final perplexity number and ask what quantisation is doing inside the model. We use Sparse Autoencoders (SAEs) on layer-3 `distilgpt2` activations because they give us a cleaner set of features to inspect than the raw dense residual stream. Quantisation makes deployment cheaper by using fewer bits, but those savings come with rounding errors that can quietly reshape the model’s internal representation.

So the main question is not just “does the model get worse?” It is: which features get hit first, how badly are they distorted at 8-, 4-, and 2-bit precision, and do those fragile features actually matter for model behaviour? We frame the damage in geometric terms—collapse, feature fragility, and rotation—so that compression becomes more than an engineering trade-off. It becomes a way to ask which internal computations are important enough to protect.

8.1 Target Model

Property	Value
Model	<code>distilgpt2</code> (HuggingFace)
Architecture	6-layer GPT-2 decoder
Hidden dimension d	768
Target layer	3 (0-indexed)
Tokeniser	<code>distilgpt2</code> BPE

9 Pipeline Setup

9.1 Activation Extraction

For the interpretability experiments, we cache layer-3 hidden states from OPENWEBTEXT rather than working directly from the full model every time. The dataset is read in streaming mode, so the pipeline never needs to keep all of OpenWebText in memory at once. The hook logic that captures these activations lives in `mechanistic_interpretability/models/hooks.py`.

The extraction setup is deliberately simple. We feed the model contiguous 128-token chunks, avoiding padding and isolated single-token samples so that each activation is still produced in a realistic local context. Before writing activations to disk, each hidden vector $\mathbf{h} \in \mathbb{R}^{768}$ is ℓ_2 -normalised to reduce trivial scale differences between tokens. The cached tensors are stored as `float16` in sharded `.pt` files under `mechanistic_interpretability/data/samples/acts_XXX`. For the assignment run, we used a 200 000-token debug pass, which is only a small slice of OPENWEBTEXT but is large enough to test the SAE and quantisation pipeline end to end.

9.2 Sparse Autoencoder Architecture

The Sparse Autoencoder is the tool we use to turn dense residual-stream activations into a more inspectable feature space. The implementation in `mechanistic_interpretability/models/sae.py` follows a Top- k design : the encoder first produces candidate feature activations, then only the largest k are allowed to pass through.

$$\mathbf{z} = \text{ReLU}(\mathbf{W}_{\text{enc}}(\mathbf{h} - \mathbf{b}_{\text{dec}})) \in \mathbb{R}^m, \quad (14)$$

$$\hat{\mathbf{z}} = \text{TopK}(\mathbf{z}, k), \quad (15)$$

$$\hat{\mathbf{h}} = \mathbf{W}_{\text{dec}}\hat{\mathbf{z}} + \mathbf{b}_{\text{dec}}, \quad (16)$$

Here, m is the SAE width and $k = \lfloor 0.1m \rfloor$ fixes the sparsity level, so only about 10% of features are active for any token. After each optimiser step, every decoder column is renormalised so that $\|\mathbf{W}_{\text{dec}}[:, i]\|_2 = 1$. This keeps feature scale from drifting and makes feature activations easier to compare.

Training hyperparameters:

Parameter	SAE-512	SAE-1024
m (bottleneck)	512	1024
k (active neurons)	51	102
Optimiser	Adam	Adam
Learning rate	10^{-4}	10^{-4}
Batch size	4096	4096
Training steps	60 000	60 000

Training uses the plain reconstruction loss $\mathcal{L}_{\text{SAE}} = \|\mathbf{h} - \hat{\mathbf{h}}\|_2^2$. No separate sparsity penalty is needed, because the Top- k operation already enforces the sparsity budget explicitly.

Table 10: Available SAE-1024 training result after 60k steps.

Model	Steps	Final Loss	L_0	Variance Explained
SAE-1024	60 000	0.0001	102.0	0.886

The available SAE-1024 log at step 60 000 reports loss = 0.0001, $L_0 = 102.0$, and variance explained = 0.886. This suggests that the Top- k routing is hitting the intended sparsity budget while reconstructing most of the activation variance.